

Final Report

Reinforcement Learning-Based Adaptive Firewall

Maria Visinescu: 530009125

Vinicius Bobato: 528004410

GitHub: <https://github.com/vmbobato/RL-Firewall>

Youtube Video: <https://youtu.be/8gSiXva4dws>

1. Introduction

Modern networks face increasingly dynamic and sophisticated cyber threats, where traditional static firewalls are insufficient for real-time defense. These static firewalls rely on manually defined rules or threshold-based policies that treat each packet independently. While effective for predictable attack signatures, these static defenses do not adapt to evolving attacker behavior, change in traffic patterns, or contextual features. As attackers become more adaptive and data network conditions shift continuously, security must also evolve to make intelligent and sequential decisions.

Reinforcement Learning (RL) is a promising approach for adaptive cybersecurity issues, because it models defenses as a sequential decision making problem. Unlike supervised machine learning classifiers that process each flow or packet in isolation, RL agents optimize long-term cumulative reward by taking into consideration how each action influences future states. In the context of a firewall, allowing or blocking a packet influences attacker persistence, interface load, and traffic features such as strike count and SYN ratios. This sequential behavior makes RL a good fit for learning adaptive policies that improve over time.

In this project, we design and evaluate a reinforcement-learning-based firewall using tabular Q-Learning, SARSA, and Deep Q Networks (DQN). Our environment models real traffic using the IoT-23 dataset [1]. The dataset contains 20 labeled malware captures executed in IoT devices, as well as 3 benign IoT devices traffic. This dataset was reduced and focused on two main malwares (C&C, Torii) and allows the agent to observe relevant flow-level features and take one of two actions: ALLOW or DENY. A reward function incentivized blocking malicious traffic while minimizing unnecessary disruption of benign traffic.

The primary objective of this project is to determine whether an RL-driven firewall can outperform static rule-based baselines in detecting malicious traffic. By comparing Q-Learning, SARSA and DQN, we investigated how each algorithm affects convergence, stability, and final detection metrics.

2. Background & Related Work

Reinforcement Learning has seen growing adoption in cybersecurity, particularly for adaptive network defense systems. Traditional firewalls rely on static rules and evaluate traffic independently, which limits their ability to respond to evolving changes in attack behavior in a network context. In contrast, RL enables sequential decision making, allowing security agents to learn policies that optimize defensive outcomes.

Prior research demonstrates the effectiveness of RL for intrusion detection, dynamic firewall rule generation, and resource allocation. This section reviews previous works that involve RL agents in the context of network security.

2.1 RL for Adaptive Firewalls and Intrusion Prevention

Several studies directly explore the use of reinforcement learning to improve firewall behavior. Cho in [2] proposes a PER-DQN-based adaptive firewall to defend IoT networks against the Okiru botnet. Their system models the defense as a Markov Decision Process with a dynamic action space that evolves based on attack stages and contextual features such as packet statistics. Their stage-aware reward function leads to faster convergence and more stable performance than traditional DQN implementations. Similarly, Ahmad [3] introduces ADF-RL, a deep reinforcement learning framework that dynamically adjusts firewall rules in real-time using a hybrid LSTM-CNN model. Evaluated using NSL-KDD and CIC-IDS2017 datasets, the system demonstrates improved detection accuracy. The authors in [4] integrate reinforcement learning with existing firewall frameworks, comparing algorithms such as DQN, PPO, A3C and ultimately showing that RL-driven rule generation improves accuracy over static rule sets.

2.2 RL for IoT and Cloud Network Defense

Beyond firewalls, RL has been applied to a wide range of IoT and cloud security contexts. The authors in [5] present an adaptive cloud security framework that uses DQN and PPO to automatically update AWS firewall rules, IAM policies, and logging configurations. By modeling cloud defense as an MDP the system dynamically responds to emerging threats and reduces policy misconfiguration. In IoT networks, the authors in [6] propose a multi-agent RL

system using MADDPG to optimize deployment of virtual network functions, such as firewalls, NATs, and load balancers. The authors in [7] evaluate SARSA and Q-Learning for defending against rank attacks in software-defined 6LoWPAN networks. Their findings show that SARSA exhibits greater robustness under adversarial conditions, outperforming Q-Learning by over 70% in packet delivery efficiency.

2.3 RL for Threat Detection and Traffic analysis

RL has also been applied for intrusion detection and protocol specific defense. Naseer in [8] presents a deep Q-Learning model for detecting DNS spoofing attacks in self-organizing IoT networks. Their system incorporates ANOVA-F feature selection and achieves a 97.5% detection accuracy, demonstrating that RL-driven IDS/IPS solutions can adapt effectively to evolving DNS-based threats.

2.4 Machine Learning for Firewall Rule Analysis

Before RL became common, machine learning was used primarily for offline firewall rule analysis. Ucar and Ozhan [9] apply ML algorithms, such as Naive Bayes, kNN, Decision Tables, and HyperPipes, to detect anomalies in larger volumes of firewall logs. Although their approach does not incorporate sequential decision-making it provides a foundation for using AI to evaluate firewall configurations.

2.5 RL for Adversarial Evasion and Security Simulation

Some research explores RL from the adversary's perspective. The authors in [10] developed "gym-waf", an OpenAI Gym environment for Web Applications Firewall (WAF) evasion, using PPO to generate adversarial payloads capable of bypassing commercial WAFs. Their work demonstrates RL's versatility and the importance of modeling sequential decision-making in both offensive and defensive contexts. The authors in [11] introduced a RL based system that simulates network attacks and defenses jointly, using their feature selection (ISSA) and multi-agent learning to optimize dynamic defensive strategies.

3. Methodology

This section describes the design of our reinforcement learning-based firewall system, including the Markov Decision Process formulation, environment construction, learning algorithms, reward function, evaluation metrics, and experimental set up. The goal is to train RL agents capable of making packet-filtering decisions based on evolving network conditions and traffic features.

3.1 Markov Decision Process Formulation

We model the firewall decision-making problem as a Markov Decision Process where the agent interacts with a sequence of packets and learns to choose actions that maximize long-term defensive performance.

State Space

Each state corresponds to a vector of discrete features derived from real IoT network traffic. The environment includes features such as:

- Flow duration
- Forward and backward packet count
- Packet length statistics
- Flow byte rate bins
- TCP flag indicators (ACK, SYN)
- Window size features
- SYN ratio

These features represent both instantaneous packet characteristics and contextual behavioral patterns. The state is discretized to enable tabular RL methods.

Action Space

The agent selects one of two actions for each incoming packet:

- ALLOW
- DENY

This action space reflects the fundamental decision made by a packet-filtering firewall.

Reward Function

We designed a reward function that reflects operational firewall priorities. It encourages the agent to aggressively block malicious traffic, minimize false negatives, avoid excessive false positives, and permit legitimate traffic.

Table 1: Reward Function

Label	Action	Reward
Malicious	DENY	+5
Malicious	ALLOW	-20
Benign	DENY	-2
Benign	ALLOW	+0.5

3.2 Environment Implementation

We implemented a custom firewall environment inspired by OpenAi Gym conventions. The environment:

- Loads traffic from CSV files
- Exposes a discrete vector for each step
- Received ALLOW/DENY decision
- Returns a reward and the next observation

This setup allows reproducible experiments and controlled comparisons between algorithms.

3.3 Learning Algorithms

In this project we evaluate three RL algorithms: Q-Learning, SARSA, and DQN.

3.3.1 Q-Learning

Q-Learning updates action-value estimates using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

3.3.2 SARSA

SARSA updates based on the action actually taken:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a_{next}) - Q(s, a)]$$

3.3.3 DQN

The Deep Q-Network extends Q-Learning by replacing the tabular Q-table with a neural network function approximator. Instead of maintaining a tabular Q-value for each state-action pair, the DQN uses a neural network to approximate action values directly from state features.

The network architecture consists of a feedforward MLP with two hidden layers of 64 units each and ReLU activations:

- Input: state vector of dimension d
- Hidden Layer 1: Linear($d, 64$) + ReLU
- Hidden Layer 2: Linear($64, 64$) + ReLU
- Output Layer: Linear ($64, 2$), producing Q-values for the two actions.

We follow the standard DQN training procedure with off-policy learning, experience replay, and a target network:

- At each timestep, the agent selects an action using ϵ -greedy policy over the current Q-network
- The transition $(s, a, r, s', done)$ is stored in a replay buffer of fixed capacity
- Once the buffer contains enough samples, the agent repeatedly samples random mini-batches and optimizes the mean-squared error between predicted Q-values and the temporal-difference target:

$$y = r + \gamma(1 - done) \max_{a'} Q_{\theta_-}(s', a')$$

Where Q_{θ_-} is a target network whose parameters are periodically synchronized with the policy network.

The loss is:

$$L(\theta) = E[(Q_{\theta}(s, a) - y)^2]$$

We used the Adam optimizer to update θ . A separate target network is maintained and updated every fixed number of environment steps to stabilize learning. Enabling the DQN agent to generalize across similar states.

3.4 Evaluation Metrics

To assess deception quality and firewall behavior, we compute

- True Positives
- False Positives
- True Negatives
- False Negatives

And derive:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$F1\ Score = 2 \left(\frac{Precision \cdot Recall}{Precision + Recall} \right)$$

Because malicious traffic is a minority class, accuracy can be misleading. Therefore, precision and recall are used as primary evaluation metrics.

3.5 Baseline Comparison

As a non-learning baseline we compare our agents to a rule-based firewall that uses hand-crafted threshold on the same flow features provided to the RL agents. This baseline mimics a traditional static firewall or IDS rule set, it does not adapt over time and always applies the same deterministic policy to every packet.

Because the policy is fixed and deterministic, multiple episodes will output the same result, so only one episode is sufficient to establish our baseline.

4. Experiments

This section describes the experimental setup used to evaluate the RL firewall, including dataset preparation, training configuration, baseline selection, and the evaluation procedure for Q-Learning, SARSA, and DQN.

4.1 Dataset & Preprocessing

We used the IoT-23 dataset [1], which contains labeled benign and malicious network traffic collected from IoT devices. For this project, we worked with three preprocessed CSV files created earlier by Vinicius: two representing malicious malware captures (Command-and-Control and Torii) and one representing benign IoT device traffic. These CSVs were merged into a single dataset, preserving the sequential order of flows to maintain realistic temporal behavior.

After merging, we applied feature selection and feature engineering to obtain the attributes most relevant to packet-level firewall decisions. The selected features were:

- Flow duration
- Total forward packets
- Total backward packets
- Mean packet length
- Flow bytes per second
- ACK flag count
- Initial forward window size
- SYN flag count
- SYN ratio (a key contextual feature representing connection attempts vs. established traffic)

Many of these features are continuous, but our RL environment uses discrete state representations to support tabular Q-Learning and SARSA. Therefore, we analyzed the distribution of each feature and partitioned it into a fixed number of bins. These bins convert

continuous network statistics into discrete categories, allowing the RL agents to operate on a compact, tractable state space.

The final output of this preprocessing pipeline is a single CSV file containing the cleaned, combined, and discretized feature vectors. This file serves as the direct input to the FirewallEnv environment, where each row corresponds to one sequential flow observation presented to the agent during training.

4.2 Agent Hyperparameters

Table 2: Q-Learning/SARSA Hyperparameters

Parameter	Value
Discount factor	0.99
e-start	1.0
e-min	0.01
e-decay	0.995
Episodes	1000

Table 3: DQN Hyperparameters

Component	Value
Hidden layers	64 -> 64 MLP
Activation	ReLU
Optimizer	Adam
Loss	MSE
Batch size	64
Replay buffer	50000
Target update freq	500

Discount factor	0.99
Episodes	1000

5. Results & Discussion

5.1 Rule-Based Agent

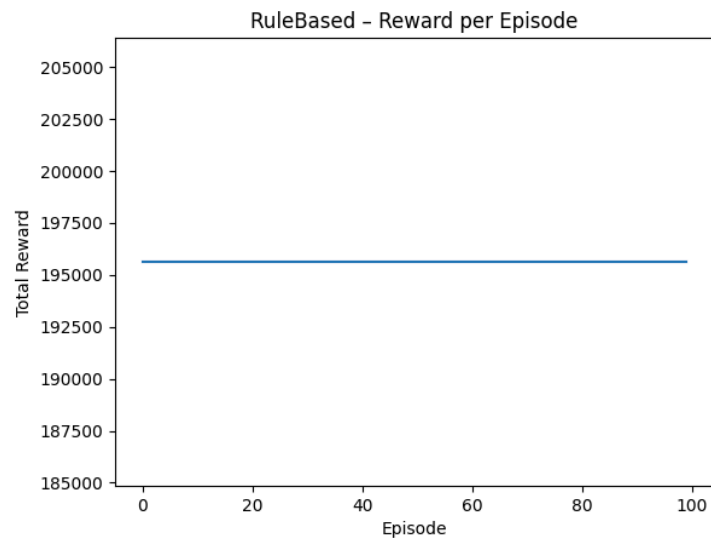


Figure 1: Rewards for Rule-Based Agent

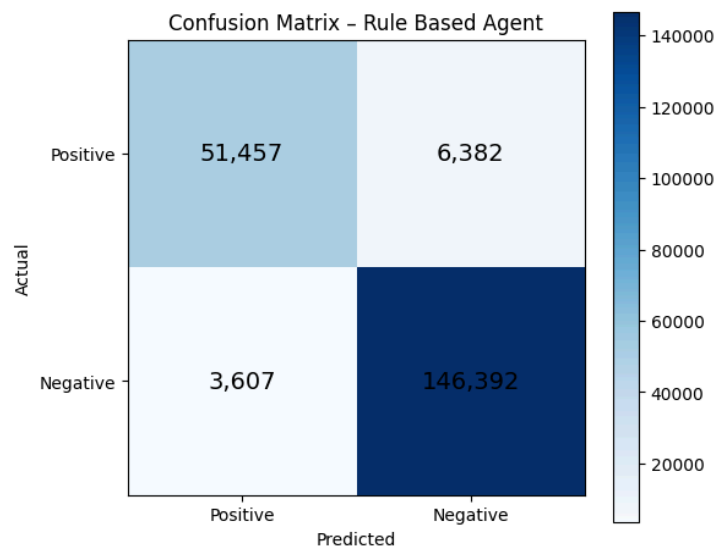


Figure 2: Confusion Matrix for Rule-Based Agent

The Rule-Based agent serves as a baseline comparison for the RL agents. Since the agent does not learn or adapt, the reward stays constant for each episode and will not improve with any additional training. The confusion matrix illustrates that while the baseline performs well on typical traffic, correctly allowing most safe traffic and blocking obvious malicious traffic, it still produces a high number of false positives and false negatives since the agent cannot capture subtle or evolving attack patterns. As a result, the Rule-Based agent can both disrupt legitimate entrance traffic and allow dangerous traffic to pass through.

5.2 Q-Learning

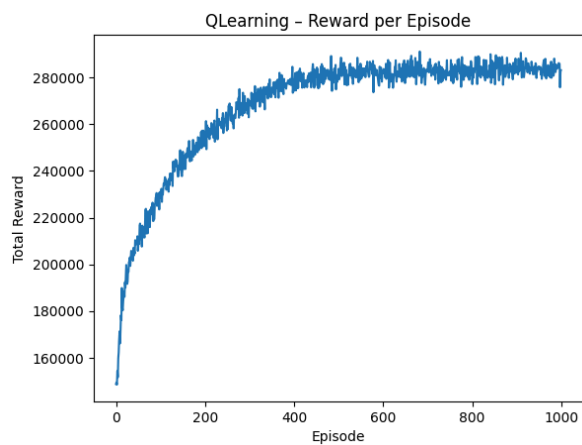


Figure 3: Q-Learning Reward Curve

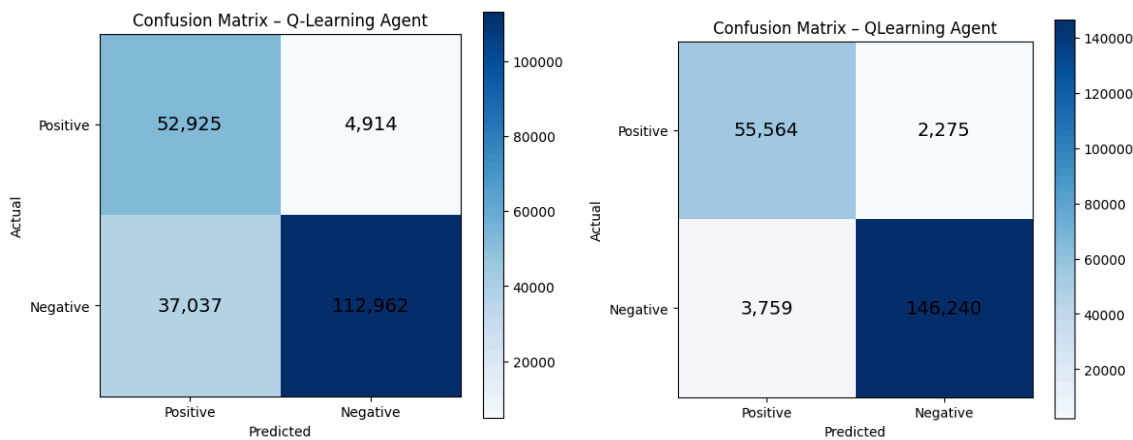


Figure 4: Q-Learning Confusion Matrices for First (left) and Last (right) Episodes

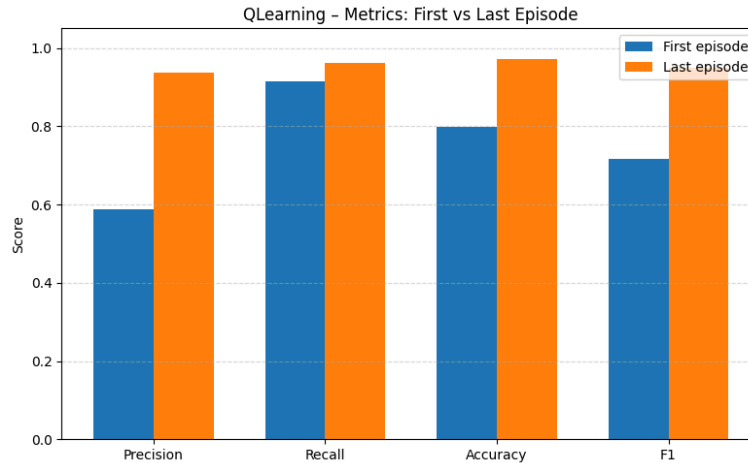


Figure 5: Q-Learning Metrics Comparison First and Last Episode

The Q-Learning agent demonstrates strong improvement over time, with rewards increasing rapidly during the first few hundred episodes and stabilizing around 400 episodes as the policy converges. The confusion matrices further highlight this progression. The agent starts with many false positives and false negatives, but by the final episode these values drop to only a few thousand, outperforming both its initial behavior and the Rule-Based agent. The metric comparison also confirms this trend. Precision, recall, accuracy, and F1 all noticeably increase from the first to the last episode as the agent becomes more effective at distinguishing malicious traffic from safe traffic.

5.3 SARSA

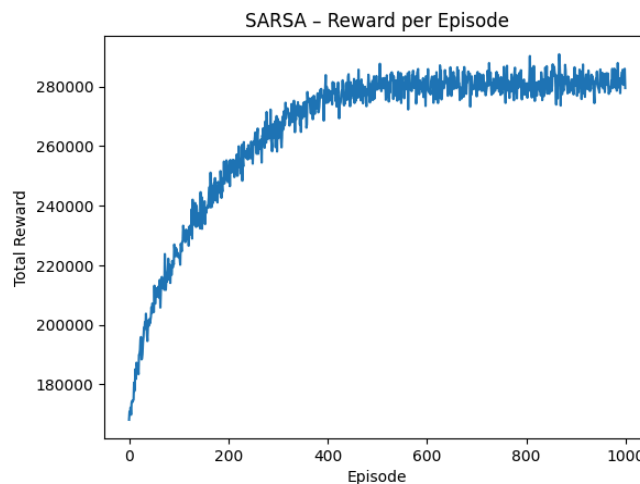


Figure 6: SARSA Reward Curve

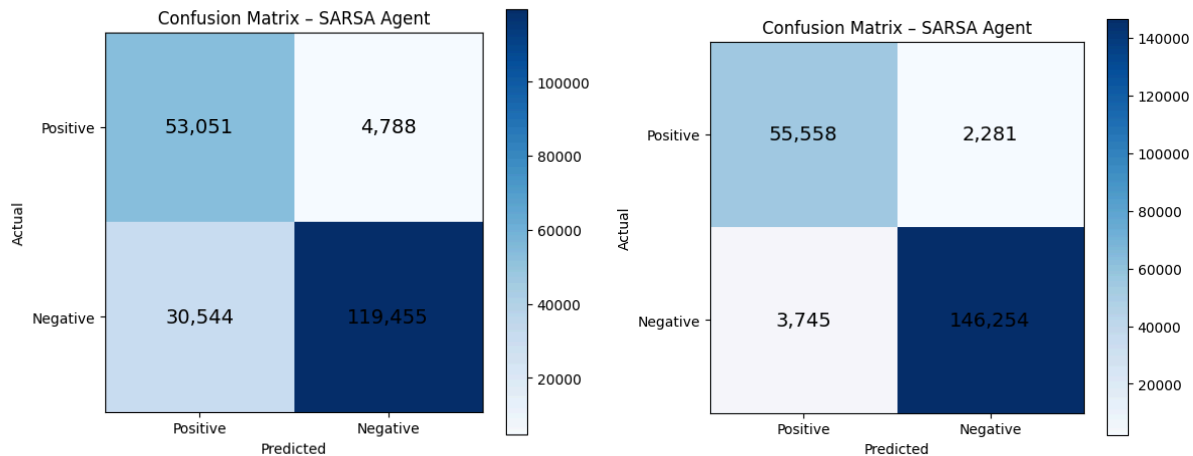


Figure 7: SARSA Confusion Matrices for First (left) and Last (right) Episodes

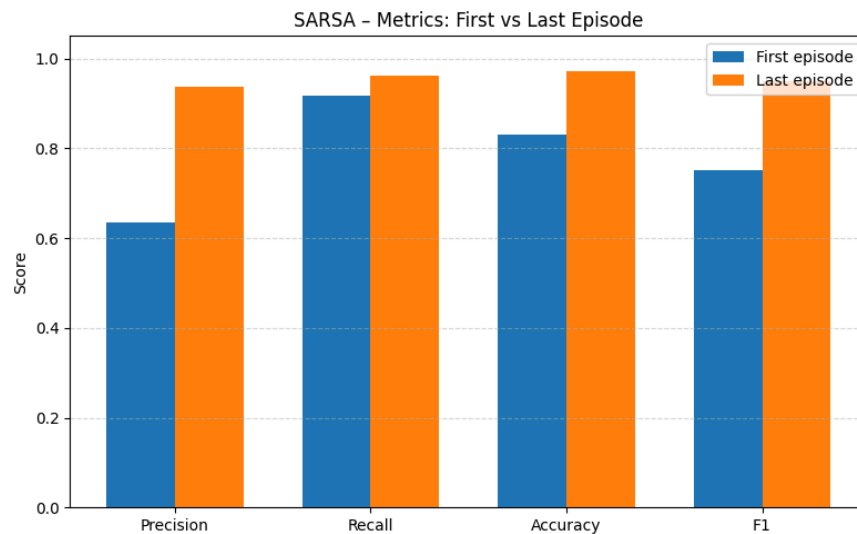


Figure 8: SARSA Metrics Comparison First and Last Episode

The SARSA agent follows a similar pattern as the Q-Learning agent also demonstrating strong improvement over time, with rewards increasing rapidly during the first few hundred episodes and stabilizing around 400 episodes as the policy converges. The confusion matrices further illustrate this process. The agent begins with many false positives and false negatives, but by the final episode these values drop to only a few thousand, outperforming the Rule-Based agent, just as the Q-Learning agent does. The metric comparison also confirms this pattern. Precision, recall, accuracy, and F1 all greatly increase from the first to the last episode as the agent becomes better at distinguishing malicious traffic from benign traffic.

5.4 DQN

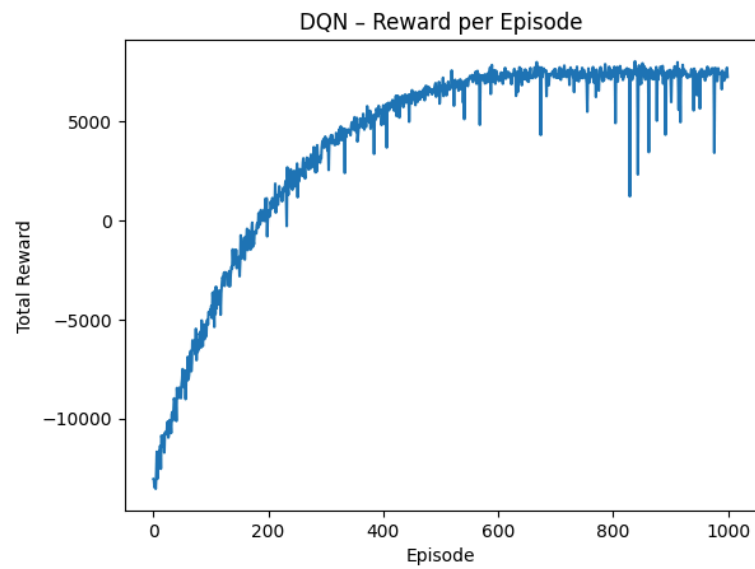


Figure 9: DQN Reward Curve

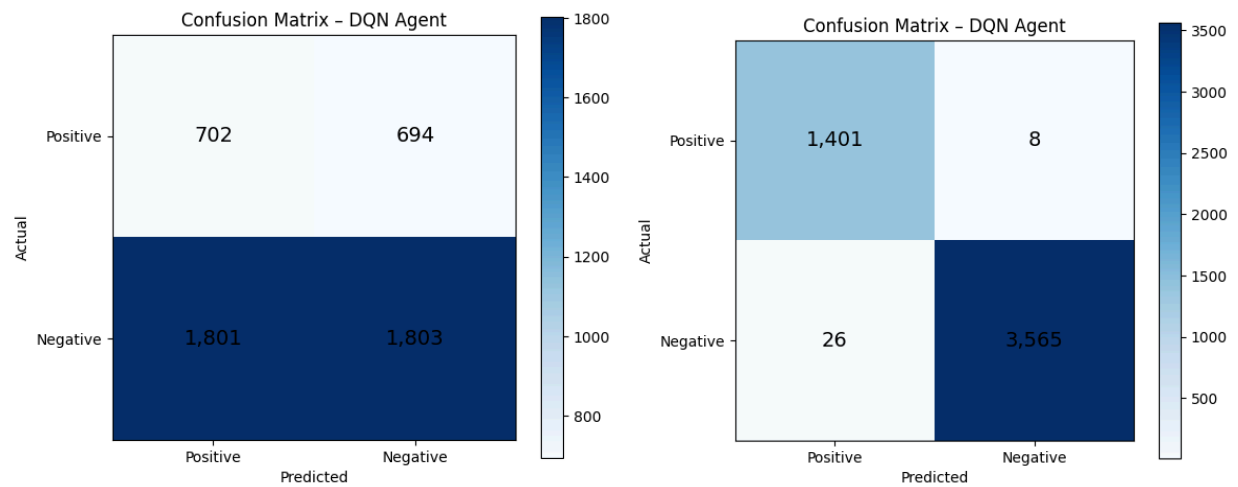


Figure 10: DQN Confusion Matrices for First (left) and Last (right) Episodes

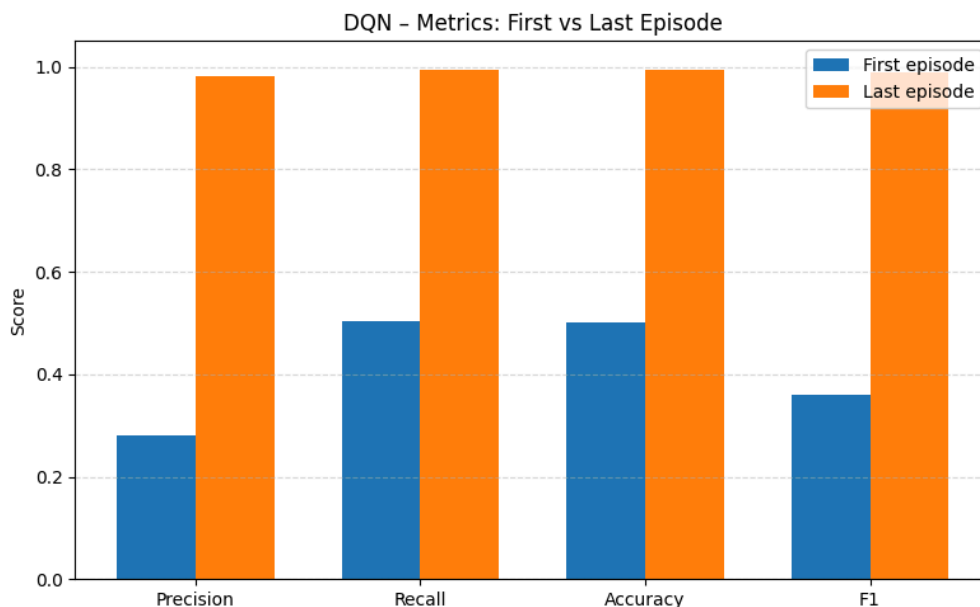


Figure 11: DQN Metrics Comparison First and Last Episode

The DQN agent showed a different learning profile compared to the Q-Learning and SARSA agents. While its training episodes produced highly negative rewards and poor detection metrics due to unstable neural network updates and extensive random exploration, performance improved steadily once the replay buffer filled and gradient updates stabilized. By the final episode, DQN achieved near perfect detection capability, with very high precision, recall, accuracy and F1-score, outperforming both Q-Learning and SARSA.

The confusion matrices illustrate this improvement: false negatives dropped from 694 from the first episode to only 8 in the final episode, and false positives went from 1801 to just 26. Although its rewards remained lower than the other agents, primarily due to having `max_steps=5000`, its policy showed to be the best in identifying complex traffic patterns.

5.5 Agents Comparison

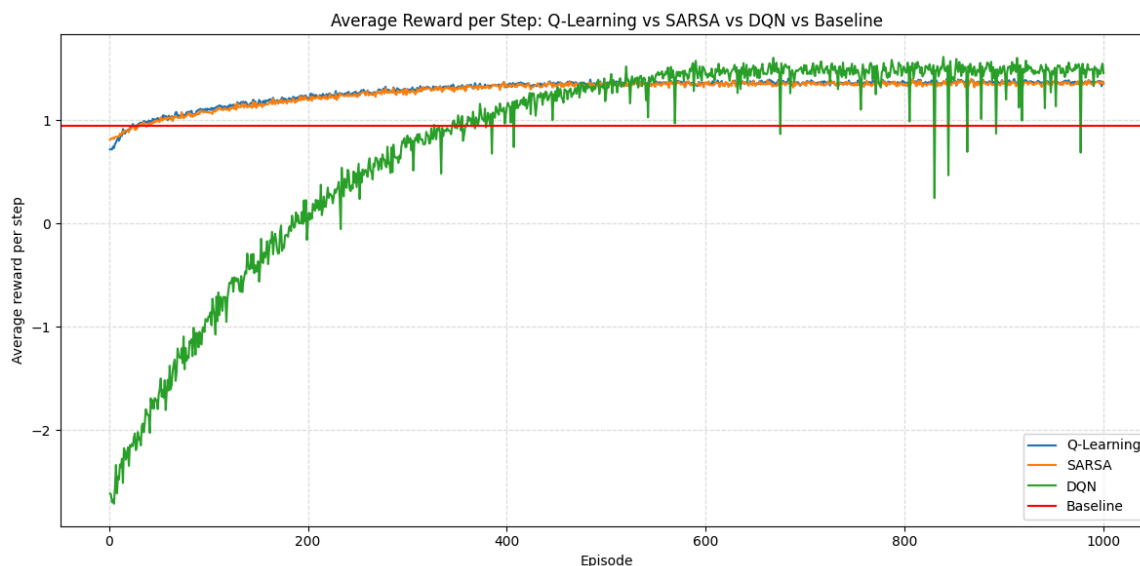


Figure 12: Average Reward Per Step Curves

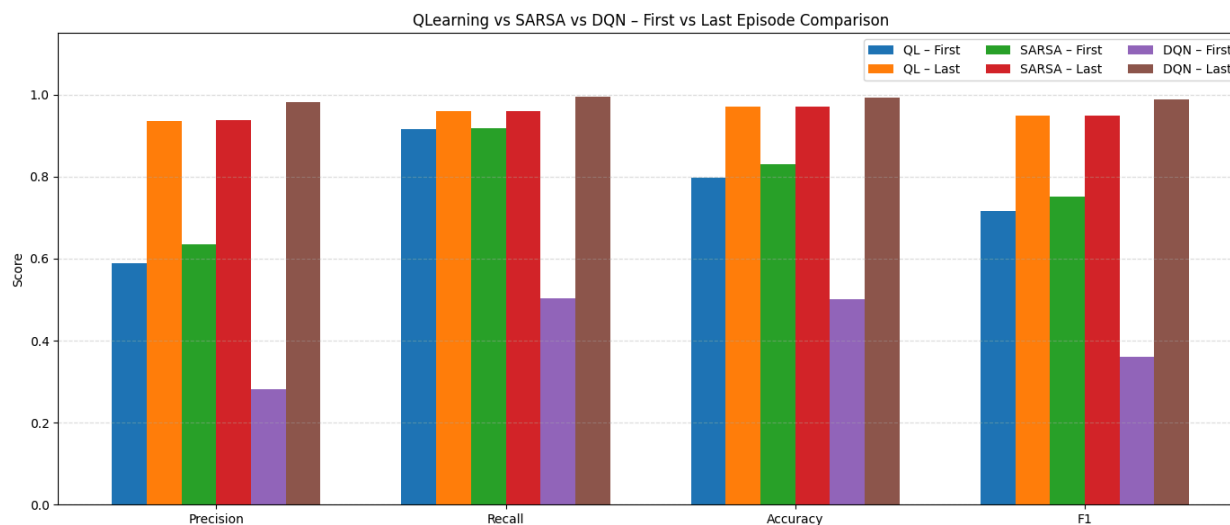


Figure 13: All Agent Metrics Comparison First and Last Episode

The average reward per step plot provides a fair comparison between the Q-Learning, SARSA, and DQN agents, despite their differing episode lengths. Q-Learning and SARSA process the entire dataset in every episode (200k+ steps), while DQN is trained with `max_steps = 5000`, resulting in much shorter episodes and therefore much smaller total rewards. Comparing raw total rewards would make DQN appear drastically worse because it interacts with fewer samples

per episode. To account for this, we normalize each episode’s total reward by dividing it by the number of steps in that episode. This produces an average reward per environment step, which reflects the decision quality of the agent, rather than the dataset size. When considered this way, DQN performs the best overall, consistently achieving the highest average reward per step, even with some variance toward the end of the training process. This matches the metric comparison, where DQN finishes with the strongest precision, recall, accuracy, and F1 out of all agents.

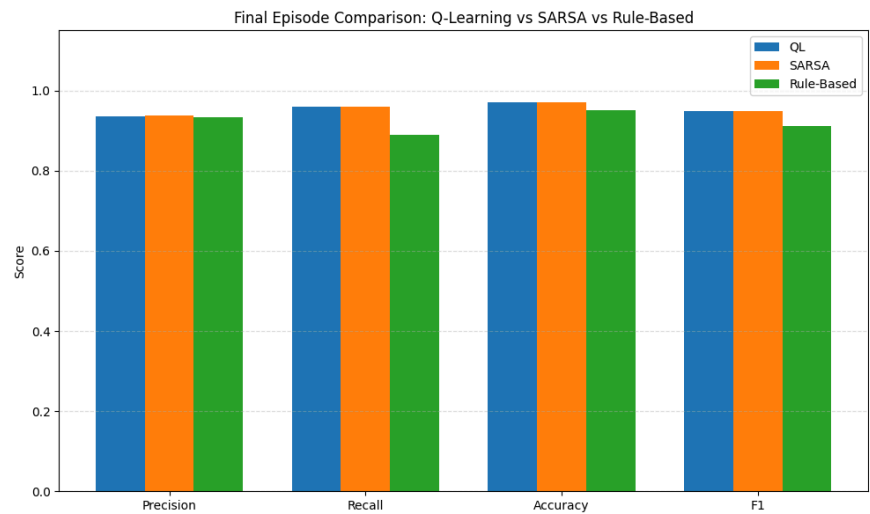


Figure 14: Q-Learning vs SARSA vs Rule-Based Metrics

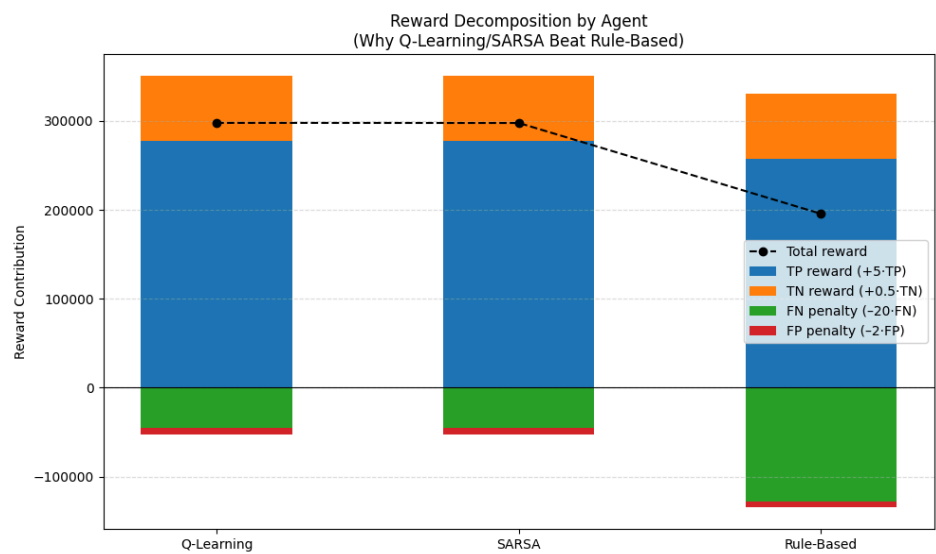


Figure 15: Agent Reward Decomposition

Although Q-Learning and SARSA achieve significantly higher total rewards, their final, precision, recall, accuracy, and F1 scores appear similar to the Rule-based baseline, seen in Figure 14. This is because the dataset is dominated by benign traffic, causing accuracy to be inflated for all agents and limiting metric sensitivity. The RL agents' improvements come from reducing false negatives and false positives on a relatively small portion of traffic, large enough to dramatically improve reward, but not large enough to drastically change global metrics as seen in Figure 15.

6. Conclusion

Our project demonstrates that reinforcement learning can provide meaningful improvements over static rule-based firewalls in detecting malicious traffic. By modeling packet filtering as a sequential decision-making problem, the RL agents learned adaptive policies that responded to both direct features like and contextual features like strike counts, SYN ratios, and window size anomalies. All three of the RL agents, Q-Learning, SARSA, and DQN, outperformed the rule-based baseline across precision, recall, accuracy, and F1-score, displaying that adaptive learning provides measurable benefits over fixed threshold rules.

Our findings show several important trends. First, even simple tabular methods like Q-Learning and SARSA can converge to strong policies when provided discretized flow features, achieving more balanced tradeoffs between false positives and false negatives compared to static rules. SARSA demonstrated slightly smoother and more stable convergence, highlighting the advantage of on-policy learning in environments where the “next action” matters. Second, while the DQN agent learned more slowly and exhibited high variance early in training, it ultimately achieved the best overall detection metrics, with near perfect recall and accuracy. This confirms that deep function approximation becomes especially powerful as the state space grows more complex.

Overall, this project shows the value of reinforcement learning for the next generation of adaptive firewalls. While rule-based systems still prove to be efficient for predictable and signature-driven attacks, RL-based policies can adapt dynamically to evolving traffic.

7. References

- [1] Sebastian Garcia, Agustin Parmisano, & Maria Jose Erquiaga. (2020). IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0) [Data set]. Zenodo. <http://doi.org/10.5281/zenodo.4743746>
- [2] H. S. Cho, J. E. Lee and S. H. Oh, "Stage-Specific Reinforcement Learning-Based Firewall for IoT Security Against Okiru Botnet," 2024 International Conference on Cyberworlds (CW), Kofu, Japan, 2024, pp. 384-386, doi: 10.1109/CW64301.2024.00085.
- [3] Ahmad, Taimoor. "AI-Driven Dynamic Firewall Optimization Using Reinforcement Learning for Anomaly Detection and Prevention." arXiv.Org, 21 May 2025, arxiv.org/abs/2506.05356.
- [4] Jha, Ashutosh Chandra. "Automated Firewall Policy Generation with Reinforcement Learning." American Academic Publisher, 24 June 2025, doi.org/10.55640/ijiot-05-01-10.
- [5] M. Saqib, D. Mehta, F. Yashu and S. Malhotra, "Adaptive Security Policy Management in Cloud Environments Using Reinforcement Learning," 2025 International Conference on Metaverse and Current Trends in Computing (ICMCTC), Subang Jaya, Malaysia, 2025, pp. 1-10, doi: 10.1109/ICMCTC62214.2025.11196240.
- [6] Y. Zhu, H. Yao, T. Mai, W. He, N. Zhang and M. Guizani, "Multiagent Reinforcement-Learning-Aided Service Function Chain Deployment for Internet of Things," in IEEE Internet of Things Journal, vol. 9, no. 17, pp. 15674-15684, 1 Sept.1, 2022, doi: 10.1109/JIOT.2022.3151134.
- [7] C. M. Moreira and G. Kaddoum, "QL vs. SARSA: Performance Evaluation for Intrusion Prevention Systems in Software-Defined IoT Networks," 2023 International Wireless Communications and Mobile Computing (IWCMC), Marrakesh, Morocco, 2023, pp. 500-504, doi: 10.1109/IWCMC58020.2023.10183144.
- [8] M. Naseer, F. Ullah, J. Ahmad, R. H. Jhaveri and T. R. Gadekallu, "Adaptive IoT Defense With Deep Q-Learning Model for DNS Spoofing Prevention in Self-Organizing Networks," in

IEEE Communications Standards Magazine, vol. 9, no. 3, pp. 64-69, Sept. 2025, doi: 10.1109/MCOMSTD.2025.3575510.

[9] Ucar, Erdem, and Erkan Ozhan. "The Analysis of Firewall Policy through Machine Learning and Data Mining - Wireless Personal Communications." SpringerLink, Springer US, 17 May 2017, link.springer.com/article/10.1007/s11277-017-4330-0.

[10] Wang, Xianbo, and Han HU. "Evading Web Application Firewalls with Reinforcement Learning." Venues, 14 Dec. 2020, openreview.net/forum?id=m5AntlhJ7Z5.

[11] Li, Ying, et al. "Application of Reinforcement Learning in Network Security in Dynamical Adjusting Defense Strategies." 27 Sept. 2024, [www.cad-journal.net/files/vol_22/CAD_22\(S7\)_2025_1-13.pdf](https://www.cad-journal.net/files/vol_22/CAD_22(S7)_2025_1-13.pdf).